

Working with Dataframes with **dplyr**

Linus Hagemann

dplyr

dplyr is a R package for working with dataframes. It's used for subsetting and manipulating data and is one of the core packages in the Tidyverse, a collection of packages used by most people in the R community. ggplot is another Tidyverse package.

```
1 library(tidyverse)
```

World Population Data

- `country`: Abbreviated country name
 - `period`: Five year period of data collection
 - `age`: Age group
 - `births`: Number of births*
 - `deaths`: Number of deaths*
 - `py.men`: Number of person-years*
 - `py.women`: Number of person-years*
- * (in thousands)

where person-year is a measure of the time contribution of each person during the period. For example, someone alive for two years during a five year period of data collection has two person-years.

Reading in a Dataframe

```
1 pop <- read_csv("data/World.csv")
2
3 pop
```

```
# A tibble: 30 × 7
```

	country	period	age	births	deaths	py.men	py.women
	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	WORLD	1950-1955	0-4	0	101090.	946911.	904909.
2	WORLD	1950-1955	5-9	0	7961.	726903.	694575.
3	WORLD	1950-1955	10-14	0	5550.	666795.	635492.
4	WORLD	1950-1955	15-19	54238.	5831.	626191.	600678.
5	WORLD	1950-1955	20-24	131939.	6649.	573518.	555221.
6	WORLD	1950-1955	25-29	128063.	6425.	508500.	507277.
7	WORLD	1950-1955	30-34	89247.	6209.	433338.	437132.
8	WORLD	1950-1955	35-39	56006.	6877.	400279.	405528.
9	WORLD	1950-1955	40-44	24327.	7995.	373771.	382444.
10	WORLD	1950-1955	45-49	5072.	8886.	326278.	333889.

```
# i 20 more rows
```

dplyr Basics

With dplyr verbs (functions)

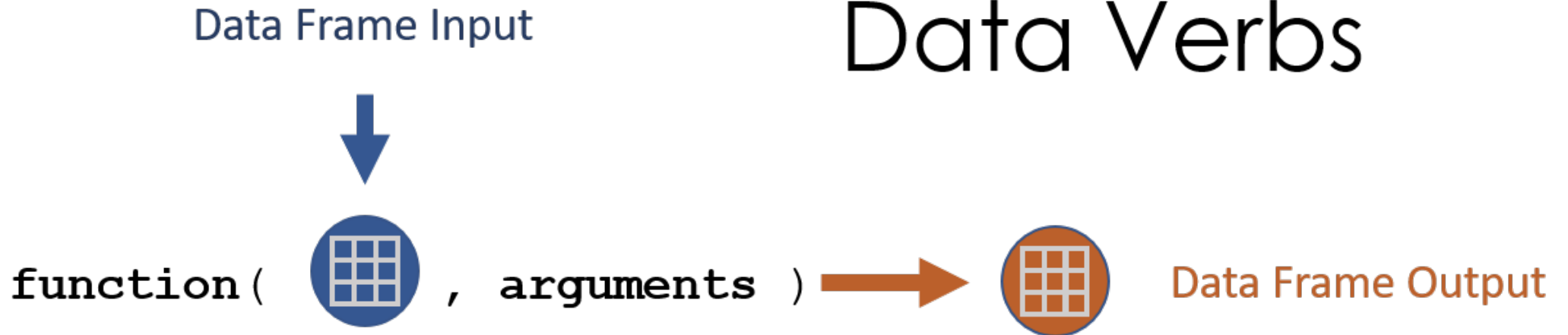
1. the first argument is always a data frame.
2. the subsequent arguments typically describe which columns to operate on using the variable names (without quotes).
3. the output is always a new data frame.

The functions are combined with the pipe operator `|>` or `%>%`.

Source: [Data Transformation in R for Data Science](#)

dplyr Basics

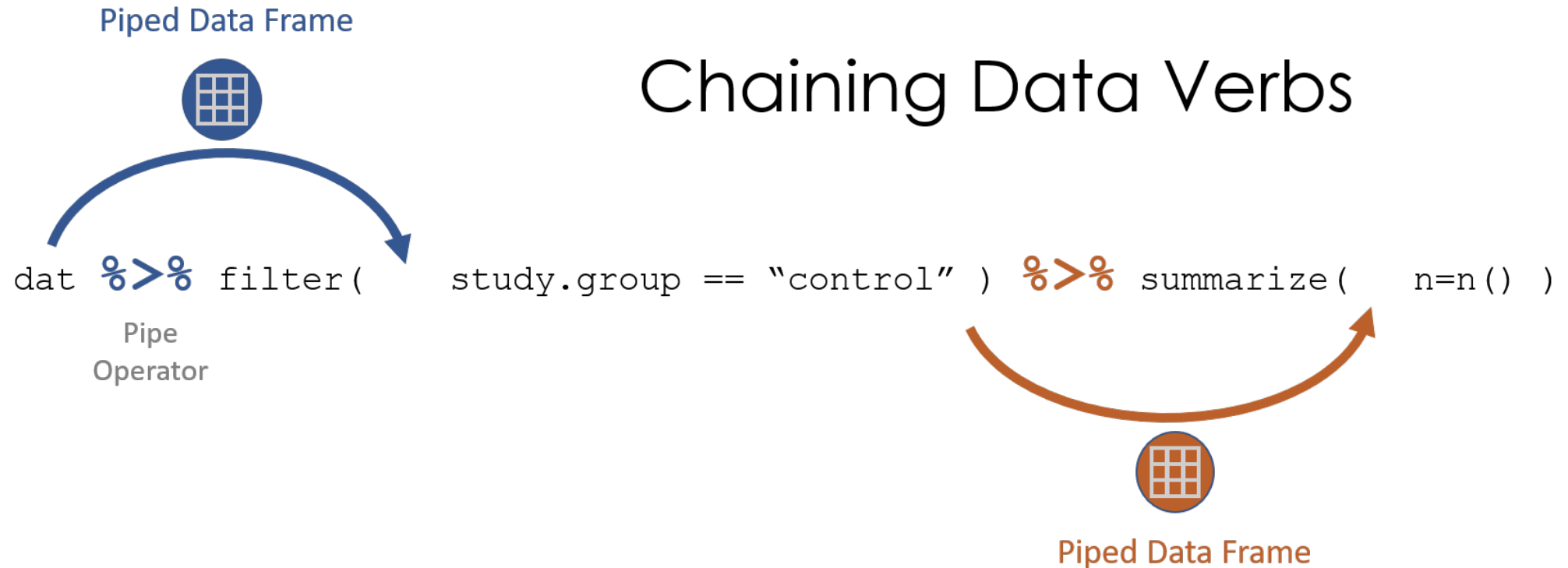
Data Verbs



The first argument of dplyr functions is a dataframe. dplyr functions return a dataframe. This makes them easy to chain together.

Source: [Data Programming for the Social Sciences](#)

The Pipe



dplyr functions can be chained together using the pipe operator (`|>` or `%>%`). The pipe operator passes the dataframe returned by a function into the next function.

Source: [Data Programming for the Social Sciences](#)

filter()

`filter()` allows us to subset rows in a dataframe. For example, if we are only interested in population data for ages 25 to 29, we can do

```
1 pop |>  
2   filter(age == "25-29")
```

```
# A tibble: 2 × 7
```

	country	period	age	births	deaths	py.men	py.women
	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	WORLD	1950-1955	25-29	128063.	6425.	508500.	507277.
2	WORLD	2005-2010	25-29	190319.	5997.	1337034.	1294853.

filter()

`filter()` can have subset rows multiple ways by separating the filters with a comma. These chunks of code do the same thing

```
1 pop |>
2   filter(age == "25-29") |>
3   filter(period == "2005-2010")
```

```
# A tibble: 1 × 7
  country period    age    births
deaths  py.men py.women
  <chr>    <chr>    <chr>    <dbl>
<dbl>    <dbl>    <dbl>
1 WORLD    2005-2010 25-29 190319.
5997. 1337034. 1294853.
```

```
1 pop |>
2   filter(age == "25-29",
3         period == "2005-2010")
```

```
# A tibble: 1 × 7
  country period    age    births
deaths  py.men py.women
  <chr>    <chr>    <chr>    <dbl>
<dbl>    <dbl>    <dbl>
1 WORLD    2005-2010 25-29 190319.
5997. 1337034. 1294853.
```

Exercise

Find the row(s) where births exceed deaths and there are more person-years for men than women.

Solution

```
1 pop |>
2   filter(births > deaths) |>
3   filter(py.men > py.women)
```

A tibble: 9 × 7

	country	period	age	births	deaths	py.men	py.women
	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	WORLD	1950-1955	15-19	54238.	5831.	626191.	600678.
2	WORLD	1950-1955	20-24	131939.	6649.	573518.	555221.
3	WORLD	1950-1955	25-29	128063.	6425.	508500.	507277.
4	WORLD	2005-2010	15-19	73195.	4043.	1593938.	1509489.
5	WORLD	2005-2010	20-24	219277.	5399.	1503388.	1442885.
6	WORLD	2005-2010	25-29	190319.	5997.	1337034.	1294853.
7	WORLD	2005-2010	30-34	114931.	6519.	1259434.	1225102.
8	WORLD	2005-2010	35-39	54976.	7238.	1210084.	1177473.
9	WORLD	2005-2010	40-44	17605.	8213.	1105937.	1082103.

arrange()

`arrange()` reorders rows based on the value of columns.

```
1 pop |>  
2   arrange(desc(births))
```

```
# A tibble: 30 × 7
```

	country	period	age	births	deaths	py.men	py.women
	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	WORLD	2005-2010	20-24	219277.	5399.	1503388.	1442885.
2	WORLD	2005-2010	25-29	190319.	5997.	1337034.	1294853.
3	WORLD	1950-1955	20-24	131939.	6649.	573518.	555221.
4	WORLD	1950-1955	25-29	128063.	6425.	508500.	507277.
5	WORLD	2005-2010	30-34	114931.	6519.	1259434.	1225102.
6	WORLD	1950-1955	30-34	89247.	6209.	433338.	437132.
7	WORLD	2005-2010	15-19	73195.	4043.	1593938.	1509489.
8	WORLD	1950-1955	35-39	56006.	6877.	400279.	405528.
9	WORLD	2005-2010	35-39	54976.	7238.	1210084.	1177473.
10	WORLD	1950-1955	15-19	54238.	5831.	626191.	600678.

```
# i 20 more rows
```

Exercise

Sort the rows where births exceed deaths and there are more person-years for men than women by deaths (descending).

Solution

```
1 pop |>
2   filter(births > deaths) |>
3   filter(py.men > py.women) |>
4   arrange(desc(deaths))
```

A tibble: 9 × 7

	country	period	age	births	deaths	py.men	py.women
	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	WORLD	2005-2010	40-44	17605.	8213.	1105937.	1082103.
2	WORLD	2005-2010	35-39	54976.	7238.	1210084.	1177473.
3	WORLD	1950-1955	20-24	131939.	6649.	573518.	555221.
4	WORLD	2005-2010	30-34	114931.	6519.	1259434.	1225102.
5	WORLD	1950-1955	25-29	128063.	6425.	508500.	507277.
6	WORLD	2005-2010	25-29	190319.	5997.	1337034.	1294853.
7	WORLD	1950-1955	15-19	54238.	5831.	626191.	600678.
8	WORLD	2005-2010	20-24	219277.	5399.	1503388.	1442885.
9	WORLD	2005-2010	15-19	73195.	4043.	1593938.	1509489.

distinct()

`distinct()` returns unique rows in the dataframe. It optionally takes column names as arguments. If supplied with column names, it returns the unique rows of the columns.

```
1 pop |>  
2   distinct(period)
```

```
# A tibble: 2 × 1
```

```
  period
```

```
  <chr>
```

```
1 1950-1955
```

```
2 2005-2010
```

count()

`count()` is like `distinct()` with named columns, but includes a column (`n`) with the number of rows in the original dataframe per unique combination.

```
1 pop |>
2   count(period)

# A tibble: 2 × 2
  period      n
  <chr>    <int>
1 1950-1955    15
2 2005-2010    15
```

Exercise

Count the number of rows in each time period where births exceed deaths and there are more person-years for men than women.

Solution

```
1 pop |>
2   filter(births > deaths) |>
3   filter(py.men > py.women) |>
4   count(period)
```

```
# A tibble: 2 × 2
  period      n
  <chr>    <int>
1 1950-1955     3
2 2005-2010     6
```

dplyr Row Exercises

Let's take a break to work through the exercises in [row-exercises.qmd](#).

select()

`select()` subsets columns in a dataframe. For example

```
1 pop |>  
2   select(period, age, births, deaths)
```

A tibble: 30 × 4

	period <chr>	age <chr>	births <dbl>	deaths <dbl>
1	1950-1955	0-4	0	101090.
2	1950-1955	5-9	0	7961.
3	1950-1955	10-14	0	5550.
4	1950-1955	15-19	54238.	5831.
5	1950-1955	20-24	131939.	6649.
6	1950-1955	25-29	128063.	6425.
7	1950-1955	30-34	89247.	6209.
8	1950-1955	35-39	56006.	6877.
9	1950-1955	40-44	24327.	7995.
10	1950-1955	45-49	5072.	8886.

i 20 more rows

rename()

`rename()` renames columns in a dataframe. For example

```
1 pop |>
2   select(period, age, births, deaths) |>
3   rename(time_period = period, age_range = age)
```

A tibble: 30 × 4

	time_period	age_range	births	deaths
	<chr>	<chr>	<dbl>	<dbl>
1	1950-1955	0-4	0	101090.
2	1950-1955	5-9	0	7961.
3	1950-1955	10-14	0	5550.
4	1950-1955	15-19	54238.	5831.
5	1950-1955	20-24	131939.	6649.
6	1950-1955	25-29	128063.	6425.
7	1950-1955	30-34	89247.	6209.
8	1950-1955	35-39	56006.	6877.
9	1950-1955	40-44	24327.	7995.
10	1950-1955	45-49	5072.	8886.

i 20 more rows

relocate()

`relocate()` changes the order of columns. By default, it moves the named columns to the front. For example

```
1 pop |>
2   select(period, age, births, deaths) |>
3   relocate(age)
```

```
# A tibble: 30 × 4
```

	age	period	births	deaths
	<chr>	<chr>	<dbl>	<dbl>
1	0-4	1950-1955	0	101090.
2	5-9	1950-1955	0	7961.
3	10-14	1950-1955	0	5550.
4	15-19	1950-1955	54238.	5831.
5	20-24	1950-1955	131939.	6649.
6	25-29	1950-1955	128063.	6425.
7	30-34	1950-1955	89247.	6209.
8	35-39	1950-1955	56006.	6877.
9	40-44	1950-1955	24327.	7995.
10	45-49	1950-1955	5072.	8886.

```
# i 20 more rows
```

mutate()

`mutate()` is used to create new columns. For example

```
1 pop |>
2   select(period, age, births, deaths) |>
3   mutate(births_deaths_ratio = births / deaths)
```

A tibble: 30 × 5

	period	age	births	deaths	births_deaths_ratio
	<chr>	<chr>	<dbl>	<dbl>	<dbl>
1	1950-1955	0-4	0	101090.	0
2	1950-1955	5-9	0	7961.	0
3	1950-1955	10-14	0	5550.	0
4	1950-1955	15-19	54238.	5831.	9.30
5	1950-1955	20-24	131939.	6649.	19.8
6	1950-1955	25-29	128063.	6425.	19.9
7	1950-1955	30-34	89247.	6209.	14.4
8	1950-1955	35-39	56006.	6877.	8.14
9	1950-1955	40-44	24327.	7995.	3.04
10	1950-1955	45-49	5072.	8886.	0.571

i 20 more rows

Exercise

Calculate the *crude birth rate* (CBR) for each row, where

$$CBR = \frac{\text{number of births}}{\text{number of person-years}}$$

Source: [Quantitative Social Science: An Introduction](#)

Solution

```
1 pop |>
2   mutate(CBR = births / (py.men + py.women))
```

```
# A tibble: 30 × 8
```

	country	period	age	births	deaths	py.men	py.women	CBR
	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	WORLD	1950-1955	0-4	0	101090.	946911.	904909.	0
2	WORLD	1950-1955	5-9	0	7961.	726903.	694575.	0
3	WORLD	1950-1955	10-14	0	5550.	666795.	635492.	0
4	WORLD	1950-1955	15-19	54238.	5831.	626191.	600678.	0.0442
5	WORLD	1950-1955	20-24	131939.	6649.	573518.	555221.	0.117
6	WORLD	1950-1955	25-29	128063.	6425.	508500.	507277.	0.126
7	WORLD	1950-1955	30-34	89247.	6209.	433338.	437132.	0.103
8	WORLD	1950-1955	35-39	56006.	6877.	400279.	405528.	0.0695
9	WORLD	1950-1955	40-44	24327.	7995.	373771.	382444.	0.0322
10	WORLD	1950-1955	45-49	5072.	8886.	326278.	333889.	0.00768

```
# i 20 more rows
```

Exercise

What row has the highest crude birth rate?

Solution

```
1 pop |>
2   mutate(CBR = births / (py.men + py.women)) |>
3   arrange(desc(CBR))
```

A tibble: 30 × 8

	country	period	age	births	deaths	py.men	py.women	CBR
	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	WORLD	1950-1955	25-29	128063.	6425.	508500.	507277.	0.126
2	WORLD	1950-1955	20-24	131939.	6649.	573518.	555221.	0.117
3	WORLD	1950-1955	30-34	89247.	6209.	433338.	437132.	0.103
4	WORLD	2005-2010	20-24	219277.	5399.	1503388.	1442885.	0.0744
5	WORLD	2005-2010	25-29	190319.	5997.	1337034.	1294853.	0.0723
6	WORLD	1950-1955	35-39	56006.	6877.	400279.	405528.	0.0695
7	WORLD	2005-2010	30-34	114931.	6519.	1259434.	1225102.	0.0463
8	WORLD	1950-1955	15-19	54238.	5831.	626191.	600678.	0.0442
9	WORLD	1950-1955	40-44	24327.	7995.	373771.	382444.	0.0322
10	WORLD	2005-2010	15-19	73195.	4043.	1593938.	1509489.	0.0236

i 20 more rows

summarise()

`summarise()` is used to calculate summary statistics for a dataframe. For example, we can find the minimum, mean, and maximum number of births in the dataset with

```
1 pop |>
2   summarise(min_births = min(births),
3             mean_births = mean(births),
4             max_births = max(births))
```

A tibble: 1 × 3

	min_births	mean_births	max_births
	<dbl>	<dbl>	<dbl>
1	0	38782.	219277.

Exercise

Find the minimum, mean, and maximum number of births among rows with more than zero births.

Solution

```
1 pop |>
2   filter(births > 0) |>
3   summarise(min_births = min(births),
4             mean_births = mean(births),
5             max_births = max(births))
```

A tibble: 1 × 3

	min_births	mean_births	max_births
	<dbl>	<dbl>	<dbl>
1	4279.	83105.	219277.

Summarise Functions

Many functions can be used with summarise, including

- `min()`
- `max()`
- `mean()`
- `median()`
- `sum()`
- `first()`: value of first row
- `last()`: value of last row
- `n()`: count of rows
- `n_distinct()`: number of distinct rows

group_by()

`group_by()` tells R to perform operations on the distinct groups defined inside `group_by()` until `ungroup()` is used. For example

```
1 pop |>
2   filter(births > 0) |>
3   group_by(period) |>
4   summarise(min_births = min(births),
5             mean_births = mean(births),
6             max_births = max(births))
```

A tibble: 2 × 4

	period	min_births	mean_births	max_births
	<chr>	<dbl>	<dbl>	<dbl>
1	1950-1955	5072.	69842.	131939.
2	2005-2010	4279.	96369.	219277.

Exercise

Calculate the crude birth rate for each period (1950-1955 and 2005-2010) across age groups by

1. Calculating the combined number of person-years for each age group for each period
2. Summing person-years and births by time period
3. Dividing births by person-years

Solution

```
1 pop |>
2   mutate(person_years = py.men + py.women) |>
3   group_by(period) |>
4   summarise(births = sum(births),
5             person_years = sum(person_years)) |>
6   mutate(CBR = births / person_years)
```

A tibble: 2 × 4

	period	births	person_years	CBR
	<chr>	<dbl>	<dbl>	<dbl>
1	1950-1955	488892.	13096959.	0.0373
2	2005-2010	674581.	33368800.	0.0202

Exercise

Let's return to the penguin dataset. Calculate the minimum, mean, median, and maximum body mass of penguins by species and island. Include the number of penguins in each sample.

Solution

```
1 penguins |>
2   group_by(species, island) |>
3   summarise(
4     min = min(body_mass),
5     mean = mean(body_mass),
6     median = median(body_mass),
7     max = max(body_mass),
8     n = n()
9   )
```

A tibble: 5 × 7

Groups: species [3]

	species	island	min	mean	median	max	n
	<fct>	<fct>	<int>	<dbl>	<dbl>	<int>	<int>
1	Adelie	Biscoe	2850	3710.	3750	4775	44
2	Adelie	Dream	2900	3688.	3575	4650	56
3	Adelie	Torgersen	NA	NA	NA	NA	52
4	Chinstrap	Dream	2700	3733.	3700	4800	68
5	Gentoo	Biscoe	NA	NA	NA	NA	124

Solution (cont)

Some values are `NA` because `NA`s propagate in R. In other words, if any values in a group are `NA`, then functions like `min()`, `max()`, and `sum()` will return `NA` by default. We can disable this behavior by including `na.rm = TRUE` in the functions.

Solution (cont)

```
1 penguins |>
2   group_by(species, island) |>
3   summarise(
4     min = min(body_mass, na.rm = TRUE),
5     mean = mean(body_mass, na.rm = TRUE),
6     median = median(body_mass, na.rm = TRUE),
7     max = max(body_mass, na.rm = TRUE),
8     n = n(),
9     # this is an easy way to get the number of NA values in a group when us
10    n_na = sum(is.na(body_mass))
11  )
```

A tibble: 5 × 8

Groups: species [3]

	species	island	min	mean	median	max	n	n_na
	<fct>	<fct>	<int>	<dbl>	<dbl>	<int>	<int>	<int>
1	Adelie	Biscoe	2850	3710.	3750	4775	44	0
2	Adelie	Dream	2900	3688.	3575	4650	56	0
3	Adelie	Torgersen	2900	3706.	3700	4700	52	1
4	Chinstrap	Dream	2700	3733.	3700	4800	68	0
5	Gentoo	Biscoe	3950	5076.	5000	6300	124	1